

Introduction to Cryptography

Pseudorandom Permutations and CPA-secure Encryption

Lei Xu

lxu12@kent.edu

Computer Science Department
Kent State University
October 2, 2026

Pseudorandom function review

Warm-up: which scheme is CPA-secure?

- Which of the following encryption schemes is CPA-secure (G is a PRG, F is a PRF)?
 - $\text{Enc}_k(m)$ chooses uniform r ; outputs $\langle r, G(r) \oplus m \rangle$
 - $\text{Enc}_k(m)$ chooses uniform r ; outputs $\langle r, F_k(r) \oplus m \rangle$
 - The one-time pad
 - $\text{Enc}_k(m)$ chooses uniform r ; outputs $\langle r, F_r(k) \oplus m \rangle$

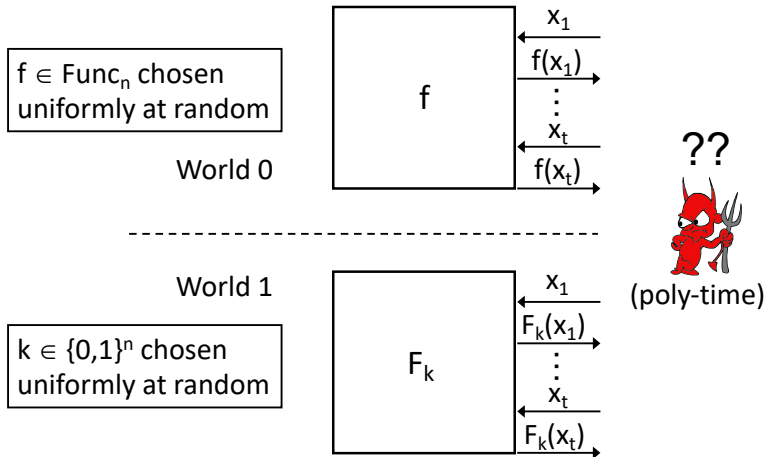
Notation

- PRG = pseudorandom generator
- PRF = pseudorandom function

Keyed functions

- Let $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be an efficient, deterministic algorithm
 - Define $F_k(x) = F(k, x)$
 - The first input is called the *key*
 - Security parameter = key length = n
- F is *pseudorandom* if F_k (for uniform k) is indistinguishable from a random function on the same domain/range

Pseudorandom functions



PRFs vs. PRGs

- PRF F immediately implies a PRG G :
 - Define $G(k) = F_k(0^n) \parallel F_k(0^{n-1}1)$, with n -bit seed k and $2n$ -bit output
 - More generally, concatenate outputs at polynomially many distinct n -bit inputs; $\parallel$ denotes concatenation
- PRF can be viewed as a PRG with random access to exponentially long output
 - The function F_k can be viewed as the $n \cdot 2^n$ -bit string $F_k(0 \cdots 0) \parallel \dots \parallel F_k(1 \cdots 1)$

Theorem: PRG–PRF equivalence

Pseudorandom generators (PRGs) exist if and only if pseudorandom functions (PRFs) exist.

Pseudorandom permutations and block ciphers

Pseudorandom permutations (PRPs)

- Let $f \in \text{Func}_n$
- f is a *permutation* if it is a bijection
 - This means that the inverse f^{-1} exists
- Let $\text{Perm}_n \subset \text{Func}_n$ be the set of permutations
- What is $|\text{Perm}_n|$?

Pseudorandom permutations

- Let F be a length-preserving, keyed function
- F is a *keyed permutation* if
 - F_k is a permutation for every k
 - F_k^{-1} , the inverse of F_k , is efficiently computable
- F is a *pseudorandom permutation* if F_k , for uniform key $k \in \{0, 1\}^n$, is indistinguishable from a uniform permutation $f \in \text{Perm}_n$
 - Even if attacker can query the function and its inverse

Note

- For large enough n , a random permutation is indistinguishable from a random function
- So in practice, PRPs are also good PRFs

Do PRFs/PRPs exist?

- They are a stronger primitive than PRGs. . .
 - . . . though they can be built from PRGs
- In practice, *block ciphers* are used as PRPs
 - Will discuss extensively later

Block ciphers

- Block ciphers are practical constructions of pseudorandom permutations
- No asymptotics: $F : \{0, 1\}^n \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$
 - $n = \text{“key length”}$
 - $\ell = \text{“block length”}$
- Hard to distinguish F_k from uniform $f \in \text{Perm}_\ell$ even for attackers running in time $\approx 2^n$

AES

- Advanced encryption standard (AES)
 - Key length = 128, 192, or 256 bits
 - Block length = 128 bits
- Will discuss details later in the course
- Available in standard crypto libraries
- No real reason to use anything else

CPA-secure encryption

Review of CPA-security: the experiment/game design

- Fix Π , $\mathcal{A}$
- Define a randomized exp't $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n)$:
 - $k \leftarrow \text{Gen}(1^n)$
 - $\mathcal{A}(1^n)$ interacts with an *encryption oracle* $\text{Enc}_k(\cdot)$, and then outputs m_0, m_1 of the same length
 - $b \leftarrow \{0, 1\}$, $c^* \leftarrow \text{Enc}_k(m_b)$, give c^* to $\mathcal{A}$
 - $\mathcal{A}$ can continue to interact with $\text{Enc}_k(\cdot)$
 - $\mathcal{A}$ outputs b' ; $\mathcal{A}$ *succeeds* if $b = b'$, and experiment evaluates to 1 in this case

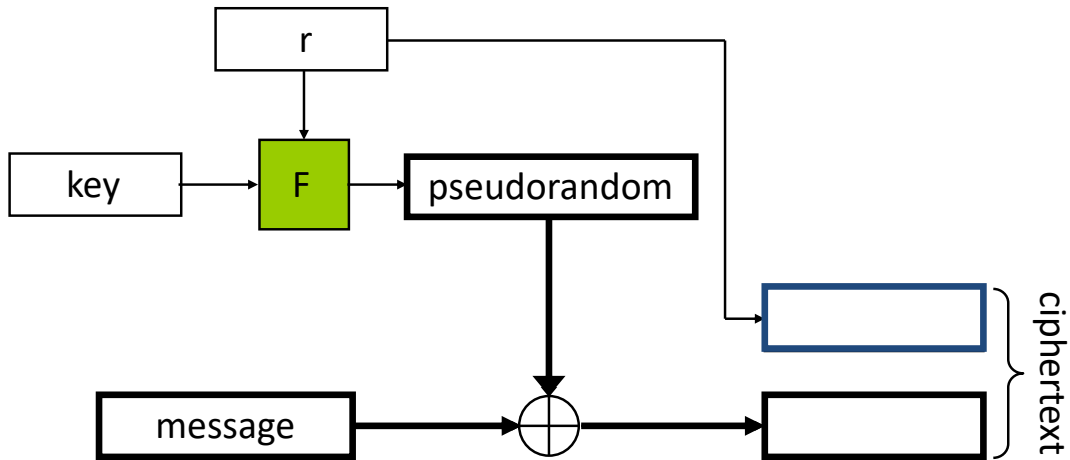
Review of CPA-security: the security definition

- Π is *secure against chosen-plaintext attacks (CPA-secure)* if for all PPT attackers $\mathcal{A}$, there is a negligible function ϵ such that $\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(n) = 1] \leq \frac{1}{2} + \epsilon(n)$

CPA-secure encryption

- Let F be a length-preserving, keyed function
- $\text{Gen}(1^n)$: choose a uniform key $k \in \{0, 1\}^n$
- $\text{Enc}_k(m)$, where $|m| = |k| = n$:
 - Choose uniform $r \in \{0, 1\}^n$ (nonce/initialization vector)
 - Output ciphertext $\langle r, F_k(r) \oplus m \rangle$
- $\text{Dec}_k(\langle r, z \rangle)$: output $z \oplus F_k(r)$
- Correctness is immediate

CPA-secure encryption



Security?

- Theorem: if F is a pseudorandom function, then this scheme is CPA-secure

Note

- The key is as long as the message. . .
- . . . but the same key can be used to securely encrypt *multiple* messages

Security proof and nonce reuse

Security?

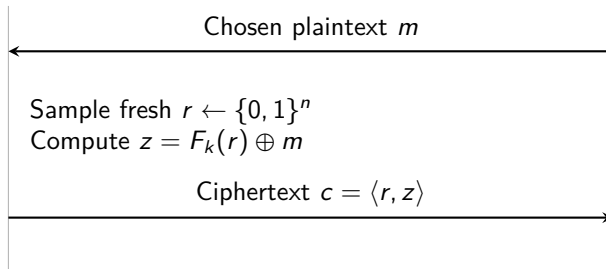
- Theorem: if F is a pseudorandom function, then this scheme is CPA-secure
- Proof by reduction. . .
 - Here: high-level intuition
 - If an efficient attacker $\mathcal{A}$ breaks the encryption scheme's CPA security with non-negligible advantage, we can construct an efficient distinguisher $\mathcal{B}$ that uses $\mathcal{A}$ to break PRF security.

Encryption oracle queries

Encryption oracle

Secret key k

Attacker $\mathcal{A}$



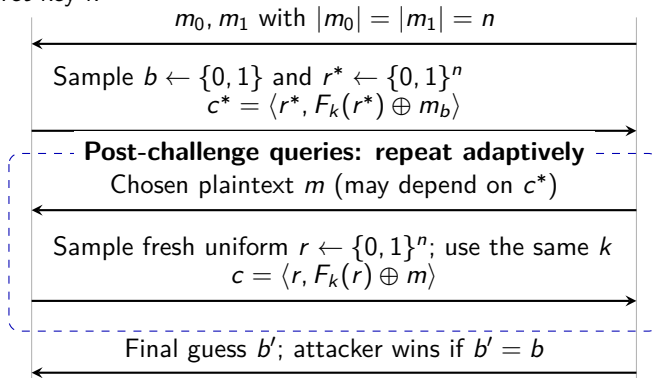
The oracle uses the same key and samples a fresh r for each query.

Challenge and post-challenge encryption queries

Challenger

Attacker $\mathcal{A}$

Secret key k



Pre-challenge encryption queries are shown on the previous slide.

Analysis: replacing the PRF with a random function

- Build a PRF distinguisher $\mathcal{B}$ that runs the CPA attacker $\mathcal{A}$.
- Its oracle O is either F_k for uniform $k \in \{0, 1\}^n$, or uniform $f \in \text{Func}_n$.
- For each encryption query m (before and after the challenge), sample $r \leftarrow \{0, 1\}^n$ and return $\langle r, O(r) \oplus m \rangle$.
- For challenge messages m_0, m_1 , sample $b \leftarrow \{0, 1\}$ and $r^* \leftarrow \{0, 1\}^n$; return $c^* = \langle r^*, O(r^*) \oplus m_b \rangle$.
- When $\mathcal{A}$ guesses b' , $\mathcal{B}$ outputs 1 (“PRF”) exactly when $b' = b$.
- Let $p_{\text{real}}(n)$ and $p_{\text{rand}}(n)$ be $\mathcal{A}$'s success probabilities when $O = F_k$ and $O = f$, respectively. PRF security gives

$$|p_{\text{real}}(n) - p_{\text{rand}}(n)| = \left| \Pr[\mathcal{B}^{F_k} = 1] - \Pr[\mathcal{B}^f = 1] \right| \leq \epsilon_{\text{prf}}(n),$$

where ϵ_{prf} is negligible; all samples above are uniform and independent.

Analysis

- We now bound $p_{\text{rand}}(n)$: $\mathcal{A}$'s success probability when the experiment uses a random function f .
- Let Win denote the event $b' = b$ in this experiment.
- There are two sub-cases
 - An encryption-oracle response, before or after the challenge, uses a nonce equal to r^* (event Repeat)
 - No encryption-oracle response uses r^* (event $\neg$ Repeat)
- Let $q(n)$ bound the total encryption-oracle queries before and after the challenge, excluding the challenge itself.

Analysis

- In the random-function experiment, by the law of total probability,

$$\begin{aligned} p_{\text{rand}}(n) &= \Pr[\text{Win}] = \Pr[\text{Win} \mid \neg\text{Repeat}] \cdot \Pr[\neg\text{Repeat}] + \Pr[\text{Win} \mid \text{Repeat}] \cdot \Pr[\text{Repeat}] \\ &\leq \Pr[\text{Win} \mid \neg\text{Repeat}] + \Pr[\text{Repeat}]. \end{aligned}$$

- $\Pr[\text{Repeat}] \leq q(n)/2^n$
 - Why?
- $\Pr[\text{Win} \mid \neg\text{Repeat}] = \frac{1}{2}$
 - Analogous to the one-time pad in this case, since $f(r^*)$ is uniform and independent of everything else
 - $p_{\text{rand}}(n) \leq \frac{1}{2} + q(n)/2^n$
- Returning to the real CPA experiment: $p_{\text{real}}(n) \leq \frac{1}{2} + q(n)/2^n + \epsilon_{\text{prf}}(n)$.
- For PPT $\mathcal{A}$, $\epsilon(n) = q(n)/2^n + \epsilon_{\text{prf}}(n)$ is negligible, as required by the CPA-security definition.

Real-world security?

- A bound is *tight up to constant factors* if an attack achieves an advantage within a constant factor of the upper bound.
- Here, the nonce-collision term $q(n)/2^n$ has this property:
 - Query $q = q(n)$ encryptions of 0^n ; each response $\langle r, z \rangle$ reveals $F_k(r) = z$.
 - Choose challenge messages $m_0 = 0^n$ and $m_1 = 1^n$. If r^* matches a stored r , recover m_b using the stored pad; otherwise guess uniformly.
- This attack's advantage over random guessing is

$$\Pr[b' = b] - \frac{1}{2} = \frac{1}{2} (1 - (1 - 2^{-n})^q) \approx \frac{q}{2^{n+1}} \quad \text{when } q \ll 2^n.$$

This matches the collision term up to a constant factor.

- What happens if a nonce r is ever reused?
- What happens to the bound if the nonce is chosen non-uniformly?

Attacks?

- If r repeats, security fails
 - *Exactly analogous* to multiple encryptions using the (pseudo)one-time pad scheme
- When r is a uniform, n -bit string, the probability of a repeat is negligible
- If r is too short, or is chosen from another distribution, repeats may happen!
 - May make scheme insecure